

# LeafLore - Project Timeline & Defense Guide

Created for Arsha (MCA Sem 3 Mini Project)

## MCA Semester 3 Mini Project: Strategic Timeline & Defense Guide

- \*Prepared for:\*\* Arsha (MCA Student)

- \*Pro

- --

### Part 1: The Presentation Strategy (Drip-Feeding)

*\*\*The Golden Rule of Student Projects:\*\* Never show a 100% completed project in the first week.*

If you  
extrem  
Softwa

#### The Rollout Strategy

You currently possess a fully functioning, production-ready codebase. To demonstrate progress, you must virtually deconstruct it and present it in weekly phases.

1. Mon

- --

### Part 2: Weekly & Monthly Progress Timeline

#### Phase 1: Planning, Schema Design & Basic Auth (July 2026)

- Goal: Show the teachers that you are setting up the foundation.\*

##### Week 1: Project Proposal & SRS

- **What to Do:** Create the Software Requirement Specification (SRS) outline. Define the scope of LeafLore.
- **What to Show:** A slide deck containing:
  - System architecture (MVC pattern: Flask controller, Jinja views, SQLAlchemy models).

- Use

- **What to Say:** "I am building LeafLore, a gamified reading habit tracker. This week I finalized the requirements, chose the

Python/Flask stack for its lightweight micro-services compatibility, and drafted the system architecture."

### #### Week 2: Database Schema & Entity-Relationship (ER) Modeling

- **What to Do:** Present the database layout.
- **What to Show:** An ER Diagram showing the relationships:
  - `User` (1-to-many) -> `Book`

- `Boo

- **What to Code/Show:** Show the Python class definitions in [models/user.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/models/user.py) and [models/book.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/models/book.py).
- **What to Say:** "I mapped out the SQLite schema using Flask-SQLAlchemy. I chose an ORM (Object-Relational Mapper) to handle transactional queries safely and ensure database portability to PostgreSQL for production."

### #### Week 3: User Authentication & Security Hardening

- **What to Do:** Enable users to register and log in.
- **What to Show:** The register/login forms.
- **What to Code/Show:** [routes/auth.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/routes/auth.py) routes and validation states.
- **What to Say:** "I implemented user authentication using Flask-Login. Passwords are encrypted using pbkdf2:sha256 hashing. I integrated Flask-WTF to prevent Cross-Site Request Forgery (CSRF) attacks."

### #### Week 4: Book CRUD & Basic Logging

- **What to Do:** Let users add books to their shelf and log pages manually.
- **What to Show:** Add Book form, Bookshelf list views, and a simple page-read logger.
- **What to Say:** "I built the basic shelf operations. Users can add books under 'want-to-read', 'reading', or 'completed' states, and log their read page count."

- --

## Phase 2: Heuristics, Analytics & API Integration (August 2026)

- Goal: Show development of statistical metrics and third-party integrations.\*

### #### Week 5: Google Books API Autocomplete

- **What to Do:** Connect the app to an external web service.
- **What to Show:** Typing a book title (e.g. "Dune") and showing autocomplete suggestions with covers.
- **What to Code/Show:** The `search\_google\_books` helper in [utils.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/utils.py) and [static/js/add\_book.js](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/static/js/add\_book.js).
- **What to Say:** "To improve user experience, I integrated the Google Books REST API. Typing a query triggers an asynchronous fetch request, fetches metadata, maps the book category to a standard library genre, and auto-fills the form."

### #### Week 6: Pacing Algorithms & Reading Velocity

- **What to Do:** Calculate average pages read.
- **What to Show:** Insights card showing "Average Daily Velocity" and "Weekly Volume".
- **What to Code/Show:** The pandas aggregation logic in [ml/analytics.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/ml/analytics.py).

- **What to Say:** `"""I wrote data aggregation algorithms using Pandas to analyze user logs. It calculates the rolling weekly volumes and average reading velocities based on active reading sessions."""`

### #### Week 7: Reading Heatmap / Calendar Grid

- **What to Do:** Display reading trends in a calendar grid.

- **What to Show:** The 53-week visual grid displaying page intensities as sprouts, stems, and flowers.

- **What to Code/Show:** `[static/js/calendar.js](file:///c:/Users/santh/OneDrive/Desktop/LeafLore_Mini_Project/static/js/calendar.js)` and the JSON api in `[routes/dashboard.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore_Mini_Project/routes/dashboard.py)`.

- **What to Say:** `"""I designed a GitHub-style activity calendar. It performs a group-by query on reading dates, maps the daily page count to distinct growth tiers (, , ), and renders cells dynamically."""`

### #### Week 8: Vocabulary Word Bank

- **What to Do:** Show the word-capturing interface.

- **What to Show:** The word card list, filters, and add word dialog.

- **What to Code/Show:** `[routes/vocabulary.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore_Mini_Project/routes/vocabulary.py)` and `[templates/vocabulary.html](file:///c:/Users/santh/OneDrive/Desktop/LeafLore_Mini_Project/templates/vocabulary.html)`.

- **What to Say:** `"""I built a Vocabulary Word Bank. Users can bookmark new words they encounter, link them to specific books, and capture the context quote to aid recall."""`

- --

## Phase 3: Gamification & Machine Learning (September 2026)

- Goal: Show the "complex" core segments of your mini-project.\*

### #### Week 9: The HTML5 Virtual Greenhouse Garden

- **What to Do:** Render the greenhouse canvas.

- **What to Show:** The canvas scene drawing ground plants, trees, and wind animations on the dashboard.

- **What to Code/Show:**

`[static/js/garden_canvas.js](file:///c:/Users/santh/OneDrive/Desktop/LeafLore_Mini_Project/static/js/garden_canvas.js)`.

- **What to Say:** `"""I built an interactive virtual greenhouse using HTML5 Canvas API. The drawing engine computes foliage density, branch scales, and tree growth states programmatically using trigonometric functions."""`

### #### Week 10: Gamified Themes, Achievements & Calm Mode

- **What to Do:** Unlock themes.

- **What to Show:** Toggling themes (Zen, Library, Autumn) and turning on Calm Mode.

- **What to Code/Show:** Flash notifications in `[utils.py]`:

`check_new_achievements()](file:///c:/Users/santh/OneDrive/Desktop/LeafLore_Mini_Project/utils.py)`.

- **What to Say:** `"""I created an achievement listener. Achieving milestones flashes badges and saves unlocks. Users can customize themes. For accessibility, I coded a 'Calm Mode' that disables canvas animation physics to support users with motion sensitivity."""`

### #### Week 11: ML Completion Predictor (Part 1 - Heuristics)

- **What to Do:** Predict if a user will complete a book.

- **What to Show:** Estimated remaining reading time and finishing likelihood on a book details card.

- **What to Code/Show:** The mathematical pacing fallback formulas in

`[ml/completion_predictor.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore_Mini_Project/ml/completion_predictor.py)`.

- **What to Say:** `"""To predict completion likelihood, I designed a predictive model. If the user is new and has sparse data (the 'cold-start' problem), the system runs an analytics heuristic that scores completion likelihood based on current velocity, elapsed days, and notes taken."""`

#### #### Week 12: ML Completion Predictor (Part 2 - Logistic Regression)

- **What to Do:** Train the classification model.
- **What to Show:** The machine learning prediction model in action.
- **What to Code/Show:** The scikit-learn Logistic Regression fitting in [ml/completion\_predictor.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/ml/completion\_predictor.py).
- **What to Say:** **"When a user has logged at least 5 completed books, the application trains a Scikit-Learn Logistic Regression model. The features extracted are: progress percentage, pages read per session, associated notes count, and focused session logs. This outputs a binary classification probability of completion."**
- --

#### Phase 4: PWA Offline Capabilities & Admin Tools (October 2026)

- Goal: Show advanced features like offline storage and administrative controls.\*

#### #### Week 13: Vocabulary Flashcard Review Mode

- **What to Do:** Show the 3D-flipping flashcard study module.
- **What to Show:** Clicking "Study Flashcards", flipping the card with a click, and scoring correct/incorrect responses.
- **What to Code/Show:** CSS 3D perspectives (``backface-visibility``, ``rotateY``) in [templates/vocabulary.html](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/templates/vocabulary.html).
- **What to Say:** **"I built an interactive Spaced Repetition study tool. It uses CSS 3D transformations to render flipping flashcards. Users can track cards they have mastered versus those that need practice."**

#### #### Week 14: PWA Support & Service Workers

- **What to Do:** Show PWA install prompts.
- **What to Show:** Running LeafLore on mobile or installing it as a desktop standalone application.
- **What to Code/Show:** The cache list and install events in [static/sw.js](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/static/sw.js) and [static/manifest.json](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/static/manifest.json).
- **What to Say:** **"I converted LeafLore into a Progressive Web App (PWA). I wrote a service worker that caches core static assets (Cache-First) and page navigations (Network-First) to support offline capability."**

#### #### Week 15: Offline Sync Queue (IndexedDB)

- **What to Do:** Mock offline logs.
- **What to Show:** Simulating offline mode in DevTools, submitting logs (amber offline banner appears), returning online, and showing auto-sync toast messages.
- **What to Code/Show:** [static/js/offline\_sync.js](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/static/js/offline\_sync.js) transactions and queue replaying.
- **What to Say:** **"If internet connection drops, the app catches it. Actions are written to IndexedDB. When connection returns, a background listener captures the event and submits the queue payload to our ``/api/offline-sync`` endpoint."**

#### #### Week 16: Admin Console & Unified Operations

- **What to Do:** Show administrative dashboards.
- **What to Show:** System diagnostic metrics (DB size, total counts) and administrative user management controls.
- **What to Code/Show:** [routes/admin.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/routes/admin.py) and [templates/admin\_dashboard.html](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/templates/admin\_dashboard.html).
- **What to Say:** **"I created a secure Admin Dashboard guarded by role checks. It aggregates operational health metrics, tracks**

database storage footprints, and lets administrators impersonate, seed, reset, or delete accounts."\*

- --

## Phase 5: Hardening, Report Compilation & Final Defense (Nov 2026)

- Goal: Secure maximum marks by showing perfect documentation, tests, and deployment.\*

### ##### Week 17: Comprehensive Testing Suite

- **What to Show:** Running `pytest` in terminal and showing 53 passing tests.

- **What to Code/Show:** [tests/test\_app.py](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/tests/test\_app.py) and GHA config [.github/workflows/ci.yml](file:///c:/Users/santh/OneDrive/Desktop/LeafLore\_Mini\_Project/.github/workflows/ci.yml).

- **What to Say:** \*"I wrote 53 automated unit and integration tests using Pytest to guarantee system reliability. I also configured a GitHub Actions CI pipeline to run audits and tests on every commit."\*

### ##### Week 18: Project Report Compilation

- **What to Do:** Write the academic project report (the "Black Book"). See Part 4 for the detailed table of contents.

- **What to Say:** \*"I compiled the final project documentation, detailing data dictionaries, data flow diagrams (DFD), system requirements, and test logs."\*

### ##### Week 19: Final Cloud Deployment

- **What to Do:** Deploy live.

- **What to Show:** Open LeafLore on your live web address (e.g. on Render/Supabase) on your phone.

- **What to Say:** \*"I configured LeafLore for production hosting on Render backed by a PostgreSQL cloud database, with security headers, Unicorn concurrency, and WhiteNoise optimization."\*

### ##### Week 20: Pre-Viva Mock Dry-Runs

- **What to Do:** Review the defense guides (Part 3) to answer examiners' questions confidently.

- --

## Part 3: Core Concept Defense Guides (How to Answer Examiners)

Examiners will probe your code to see if you wrote it. Prepare for these common questions:

### Question 1: "Explain how your ML prediction works under the hood."

- **The Trap:** If you say "it just predicts it," you will lose marks. You must show mathematical knowledge.

- **How to Answer:**

1. \*"We have a dual-engine architecture: a Heuristic Fallback Engine and a Logistic Regression Engine."\*

2. \*"If  
adjust

Question 2: "How does the HTML5 Garden Canvas render plants dynamically?"

- **The Trap:** They might ask if it's a static image or a video.
  - **How to Answer:**
1. \*It is rendered programmatically on an HTML5 <canvas> element in real-time.\*

2. \*If  
we de  
and M

Question 3: "Explain how your PWA supports offline sync without losing data."

- **How to Answer:**
1. \*We use a custom service worker (sw.js) that caches static assets using a Cache-First strategy and HTML routes using a Network-First fallback strategy.\*

2. \*If  
reques

Question 4: "Why did you use CSS 3D Transforms instead of Three.js for the bookshelf?"

- **How to Answer:**
1. \*Using a massive WebGL engine like Three.js adds several megabytes of weight to the application, slowing page load times, which violates mobile-first PWA guidelines.\*

2.  
[templ  
proper  
spines

Part 4: What to Show, What NOT to Show/Say

| [DONE] What to SHOW & SAY                                                            | [NOT DONE] What NOT to Show or Say                                                                        |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Say you built this incrementally over 5 months.                                      | Never say you finished the code in June or used AI to generate blocks.                                    |
| Show your ER Diagram and Database design first.                                      | Never show the Admin Dashboard on day 1. It makes the app look complete too early.                        |
| Say SQLite is used for local development, and PostgreSQL for production.             | Never say "SQLite is only a toy database" (teachers hate this). Explain SQLite's concurrent limitations.  |
| Show terminal outputs (pytest runs, database migrations). It looks highly technical. | Never say "It works on my machine but not on the server." Verify production hosting early.                |
| Explain the math of your fallback heuristic and Logistic Regression formulas.        | Never say "I just imported scikit-learn and it worked." You must explain features and training variables. |

- --

## Part 5: MCA Project Report Structure (The Black Book)

For your final evaluation, you will need a physical/PDF project report. Format it exactly like this to satisfy academic standards:

1. Title Page & Certificates (College-approved format)

2. Abs